Collections: Lists, Sets & Strings

Class 5 — In-Class Work-Along Sheet (working title · built iteratively)

Name: _____ Section: _____ Date: _____

How to use this sheet: Write your prediction **first**, then run it. A wrong guess you then fix sticks far better than a right answer handed to you. Keep your **Terminal** open throughout.

Part 1 — Three Ways to Hold Data: Lists, Sets & Strings

Q1. *(The hook — predict ALL three outputs before running anything.)*

```
names = "Python"
scores = [95, 87, 92]
tags = {"Python", "AI", "Beginner"}

print(names[1])
print(scores[1])
print(tags[1])
```

(a) Predict all three: _____ · _____ · _____

(b) Run all three. Two work; one crashes. Which one? _____

(c) The error says: `_____`

In one line — what does this tell you about how sets store data?

_____

Q2. *(Three containers, three types.)*

Run this:

```
print(type("hello"))
print(type([1, 2, 3]))
print(type({1, 2, 3}))
```

(a) Fill in: `"hello"` → `_____` · `[1, 2, 3]` → `_____` · `{1, 2, 3}` → `_____`

(b) **Ordered** means Python keeps track of _____, so you can ask "what's at position 2?"
Unordered means Python only tracks _____ — just "is this item here?"

(c) Which two of the three types are ordered? _____ and _____

(d) Footgun: `{}` (empty curly braces) gives a `dict`, not a `set`. Run both to confirm:

```
print(type({}))      # _____
print(type(set()))   # _____
```

To make an empty set you must write: _____

Q3. *(Mutable vs immutable — predict FIRST, then run.)*

Block A:

```
s = "hello"
s[0] = "H"
print(s)
```

Block B:

```
lst = [10, 20, 30]
lst[0] = 99
print(lst)
```

(a) Block A prediction: _____ Block B prediction: _____

(b) Run both. Block A → error: `_____` Block B → prints: _____

(c) Fill in:

- A `str` is **immutable** — its characters _____ be changed in place after creation.
- A `list` is **mutable** — you _____ change individual elements.
- One reason immutability matters: _____

Q4. *(Sets eat duplicates — and elements must be hashable.)*

(a) Predict both, then run:

```
print({1, 2, 2, 3, 3, 3})
print([1, 2, 2, 3, 3, 3])
```

Set output: _____ List output: _____

(b) A visitor log `{101, 202, 101, 303, 202}` as a set becomes: _____

(c) Now run each `add()` call one at a time — note which one crashes:

```
s = {1, 2, 3}
s.add(42)
s.add("hello")
s.add((10, 20))
s.add([1, 2])
```

Which call crashes? _____ Error: `_____`

(d) Fill in — **hashable** means Python can compute a stable _____ (fingerprint) for the object:

- `42` (int) → _____ hashable
- `"hello"` (str) → _____ hashable
- `(10, 20)` (tuple of ints) → _____ hashable
- `[1, 2]` (list) → **not** hashable, because a list is _____ and its contents can change.

(e) Rule of thumb: immutable built-in types are hashable; mutable built-in types are not. One wrinkle — try this:

```
print(hash((1, 2, 3)))       # _____  (works)
print(hash((1, [2, 3])))     # _____  (what happens?)
```

Why does the second one fail even though a tuple is immutable?

_____

Q5. *(`in` works on all three — predict ALL six, then run.)*

```
print('y' in "Python")
print(87 in [95, 87, 92])
print("AI" in {"Python", "AI", "Beginner"})
print('z' in "Python")
print(100 in [95, 87, 92])
print("SQL" in {"Python", "AI", "Beginner"})
```

(a) Predict all six: _____ · _____ · _____ · _____ · _____ · _____

(b) Run. The `in` operator always returns type: _____

(c) Planted seed — just your intuition: for checking membership in a collection of **one million** items, which type would be fastest — `str`, `list`, or `set`? _____ One-line reason:

_____

Q6. *(Choosing the right container — four scenarios.)*

Choose `str`, `list`, or `set` for each, and justify in one sentence:

(a) A social media app stores posts in the order they were published, and must show post #47. → _____ Because: _____

(b) A product name `"MacBook Pro 14-inch"` stored in memory — never needs character changes. → _____ Because: _____

(c) A system deduplicates user IDs to count unique visitors to a page. → _____ Because: _____

(d) A quiz stores 10 round scores in order, and the last score needs to be updated after a correction. → _____ Because: _____

Q7. *(Recall — books closed.)*

- Three collection types from this part: _____ · _____ · _____
- Their literal syntax: _____ · _____ · _____
- Ordered means _____; unordered means _____
- Immutable means _____; mutable means _____
- Set elements must be _____. Rule of thumb: _____ built-in types are hashable; _____ are not
- Empty set (not dict): _____
- `in` returns type _____. For large data, fastest membership check → _____

Q8. *(Python vs Java/C++: heterogeneous containers.)*

In Java/C++, every element in an array must be the same type: `int[] arr = {1, 2, 3}` — a string would be rejected at compile time. Python has no such restriction. Predict the outputs, then run:

```
mixed = [42, "hello", True, 3.14, [1, 2, 3]]
print(len(mixed))
print(type(mixed[0]))
print(type(mixed[1]))
print(type(mixed[4]))
```

(a) Outputs: _____ · _____ · _____ · _____

(b) Fill in:

- Python lists are **heterogeneous** — they can hold elements of _____ types simultaneously.
- Java/C++ arrays are **homogeneous** — every element must share the _____ type.

(c) Python's flexibility is powerful — but name one risk of mixing types carelessly in a list:

_____

Q9. *(Containers inside containers.)*

Python containers can hold any object — including other containers. No limit on nesting depth.

(a) Nested list — a 3×3 matrix. Predict all three, then run:

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

print(matrix[1][2])
print(matrix[0][0])
print(matrix[2][1])
```

`matrix[1][2]` reads as: row _____, column _____ → _____ `matrix[0][0]` → _____ `matrix[2][1]` → _____

(b) Set of tuples — unique GPS coordinates. Predict, then run:

```
coords = {(0, 0), (1, 2), (3, 4), (1, 2)}
print(coords)
print(len(coords))
```

How many elements after deduplication? _____ *(Note: display order may vary — sets are unordered.)*

Why can tuples live in a set but lists cannot? _____

(c) List of sets — an article tag system. Predict all three, then run:

```
article_tags = [
    {"Python", "AI"},
    {"Python", "Web"},
    {"AI", "ML"},
]
print(article_tags[0])
print("AI" in article_tags[1])
print(len(article_tags))
```

Predictions: _____ · _____ · _____ *(set display order may vary)*

Why is a list of sets a natural fit here — why not a list of lists?

_____

(d) Challenge — read this structure and predict:

```
data = [
    ("Alice", {95, 87, 92}),
    ("Bob",   {78, 85, 90}),
]
print(data[0][0])
print(data[1][1])
print(88 in data[0][1])
```

`data[0][0]` → _____ `data[1][1]` → _____ *(order may vary)* `88 in data[0][1]` → _____

In plain English, what does this data structure represent? _____

What type is `data`? _____ What type is `data[0]`? _____ What type is `data[0][1]`? _____

Part 2 — Debugging Lab: RapidKart Technologies

⟳ **Spaced retrieval in action.** This lab spans Classes 1–5. Every bug you find means you recalled the rule it broke. When you identify a bug, note *which class* taught the rule — that act of reaching back is the learning.

The story: RapidKart Technologies, a fictional logistics-tech startup, recently open-sourced part of their internal analytics pipeline on GitHub. Their senior engineer flagged the function `audit_inventory()` with

a one-line comment: *"passes basic tests but behaves wrong in edge cases — needs a thorough review before production."*

The function scans the day's product feed and produces a quality report. You've been asked to do the code review.

One new thing you'll see in this code — a dictionary. A dictionary (`dict`) maps keys to values: `p = {"sku": "A101", "stock": 5, "price": 299.99}`. You access a value by its key: `p["stock"]` gives `5`. Think of it as a row in a spreadsheet — each column name is a key. We'll teach dicts fully in a later class; for now, just read `p["stock"]` as "the stock field of product `p`."

The code contains exactly 8 bugs. Some crash the program immediately. Some produce silently wrong output — the program runs, but the answer is wrong. One is a poor design choice that causes problems at scale. Find them all.

```
import math

# ──────────────────────────────────────────────────────────────
# RapidKart Technologies — Inventory Audit Module
# File: quality_checker.py  |  Author: Rohan M.  |  Status: Review
# ──────────────────────────────────────────────────────────────

ADMIN_BIT     = 0b100   # permission: full audit access (bit 2)
WAREHOUSE_BIT = 0b010   # permission: warehouse view   (bit 1)
VIEWER_BIT    = 0b001   # permission: read-only        (bit 0)


def audit_inventory(products, user_flags):
    """
    products  : list of dicts — each has 'sku' (str), 'name' (str),
                'stock' (int), 'price' (float), 'premium' (bool)
    user_flags: int, bit-encoded permissions for the calling user
    Returns   : set of flagged SKU IDs requiring manual review
    """

    # ── 1. Verify the calling user has admin access ──────────
    if user_flags & ADMIN_BIT == 1:
        print("Full audit access granted")
    else:
        print("Limited access — read-only mode")

    # ── 2. Build the report header ───────────────────────────
    total_skus = len(products)
    header = "RapidKart Audit — SKUs scanned: " + total_skus

    # ── 3. Track unique SKUs seen across warehouses ──────────
    seen_skus = []
    for p in products:
        seen_skus.append(p["sku"])

    # ── 4. Flag products with invalid stock or price data ────
    flagged = {}
    for p in products:
        if p["stock"] < 0 or p["price"] < 0:
            flagged.add(p["sku"])

    # ── 5. Spot-check delivery fee rate ──────────────────────
    base_fee      = 0.1
    surcharge     = 0.2
    combined_rate = base_fee + surcharge
    if combined_rate == 0.3:
        print("Delivery fee rate: OK")
    else:
        print("Delivery fee rate: MISMATCH — investigate!")

    # ── 6. Compute average product price ─────────────────────
    total_value = sum(p["price"] for p in products)
    avg_price   = total_value // len(products)
    print(f"Average price: Rs.{avg_price:.2f}")

    # ── 7. Capitalise brand name for the report ──────────────
    brand_name = "rapidkart"
    brand_name[0] = brand_name[0].upper()
    print(f"Brand: {brand_name}")

    # ── 8. Flag out-of-stock items as removal candidates ─────
    for p in products:
        is_premium = p.get("premium", False)
        if p["stock"] == 0 or not is_premium:
            flagged.add(p["sku"])
```

```
    print(header)
    print(f"Unique SKUs tracked: {len(seen_skus)}")
    return flagged
```

Q10. *(Read first — understand the intent before hunting bugs.)*

(a) What is `ADMIN_BIT = 0b100` in decimal? Show your working: _____

(b) In plain English, what does `user_flags & ADMIN_BIT` check? _____

(c) Why should `flagged` be a `set` rather than a `list`? _____

(d) Section 5 checks whether `combined_rate == 0.3`. If the fee rate is correct, what output do you expect? _____

Q11. *(Bug Hunt — find all 8. For each: copy the buggy line, classify it, trace the root cause, write the fix.)*

Hint if stuck: look for bugs from these categories — type coercion · float precision · bitwise precedence · logical operators · string immutability · `{}` footgun · wrong container for the job · integer division. Each category appears exactly once.

Bug 1

- Buggy line: _____
- Crash / wrong output / design: _____
- Class that taught the rule broken: Class _____
- Root cause: _____
- Fixed line: _____

Bug 2

- Buggy line: _____
- Crash / wrong output / design: _____
- Class: _____ Root cause: _____
- Fixed line: _____

Bug 3

- Buggy line: _____
- Crash / wrong output / design: _____
- Class: _____ Root cause: _____
- Fixed line: _____

Bug 4

- Buggy line: _____
- Crash / wrong output / design: _____
- Class: _____ Root cause: _____
- Fixed line: _____

Bug 5

- Buggy line: _____
- Crash / wrong output / design: _____
- Class: _____ Root cause: _____
- Fixed line: _____

Bug 6

- Buggy line: _____
- Crash / wrong output / design: _____
- Class: _____ Root cause: _____
- Fixed line: _____

Bug 7

- Buggy line: _____
- Crash / wrong output / design: _____
- Class: _____ Root cause: _____
- Fixed line: _____

Bug 8

- Buggy line: _____
- Crash / wrong output / design: _____
- Class: _____ Root cause: _____
- Fixed line: _____

Q12. *(↻ Spaced retrieval — books closed. No code needed.)*

(a) Use the precedence ladder to explain WHY `user_flags & ADMIN_BIT == 1` never grants access even when `user_flags = 0b110`. Write what Python actually evaluates step by step.

(b) Convert `0b100` to decimal. Show each step: _____

(c) `0.1 + 0.2 == 0.3` returns `False` in Python. Explain why in two sentences. What is the correct comparison to use instead? _____

(d) `899.97 // 3` vs `899.97 / 3` — what does each return? Which is correct for computing an average price? _____

(e) Why does `brand_name[0] = 'R'` crash? Which property of strings causes this? _____

Q13. *(Synthesis)*

(a) List the three bugs that cause an immediate crash (TypeError or AttributeError): 1. _____ 2. _____ 3. _____

(b) The other five produce **silently wrong results** — no error, just wrong data quietly going through the system. Which one do you think is most dangerous in a live production system? Justify in two sentences.

(c) Bug 7 (`seen_skus = []`) never crashes — it just gives a wrong unique-SKU count. Why is this kind of "design bug" sometimes harder to catch than a crash bug? *(One sentence.)*

Part 3 — Strings: Indexing, Slicing, Methods & Tuples

Q14. *(Indexing — predict ALL outputs before running.)*

```
s = "Python"

print(s[0])
print(s[1])
print(s[-1])
print(s[-2])
print(s[3])
```

(a) Predictions: _____ · _____ · _____ · _____ · _____

(b) Run and confirm. Fill in the index diagram:

```
  P    y    t    h    o    n
[ __ ][ __ ][ __ ][ __ ][ __ ][ __ ]   ← positive indices
[ __ ][ __ ][ __ ][ __ ][ __ ][ __ ]   ← negative indices
```

(c) `s[-1]` gives the last character without knowing the length. Rule: negative index `-n` is the same as index `len(s) - ____`.

(d) Predict what `s[10]` produces, then run: _____

Q15. *(Slicing — go deep.)*

```
s = "Programming"
```

(a) Basic and defaults — predict each, then run:

```
s[3:7]    →  _________
s[:4]     →  _________
s[4:]     →  _________
s[-3:]    →  _________
s[:-2]    →  _________
s[:]      →  _________
```

(b) Off-by-one rule: `s[1:4]` returns _____ characters, not 4. Complete: slicing **includes** index `start` and **excludes** index _____.

(c) Step parameter — predict each, then run:

```
s[::2]    →  _________   (takes every ______ character)
s[::-1]   →  _________   (what does this do to the string?)
s[1::2]   →  _________
```

(d) ○ Strings are immutable. Predict what happens, then run:

```
s[2:5] = "xyz"
```

Error: _____ . Slicing *reads* a range but cannot _____ one.

Q16. *(`len()` and `in` / `not in` on strings.)*

```
text = "hello world"

print(len(text))
print('o' in text)
print('ow' in text)
print('xyz' in text)
print('world' in text)
print('World' in text)
```

(a) Predict all six: _____ · _____ · _____ · _____ · _____ · _____

(b) Run and verify. ↻ In Part 1, `in` on a *set* checked for a member element. On a string, `in` checks for a _____ — it can be one character or many. This is called a _____ check.

(c) Why does `'World' in text` return `False`? _____

Q17. *(String methods — each returns a NEW string; the original is unchanged.)*

(a) Case conversion — predict the third print, then run all three:

```
s = "Hello World"
print(s.upper())
print(s.lower())
print(s)
```

After calling `.upper()`, `s` itself is: _____ . Why? _____

(b) Strip — predict, then run:

```
raw = "   hello world   "
print(raw.strip())
print(raw.lstrip())
print(raw.rstrip())
print(len(raw), len(raw.strip()))
```

Outputs: _____ · _____ · _____ · _____ and _____

(c) Split — the workhorse of text processing. Run each block and fill in:

```
print("  hello   world  ".split())
```

Output: _____ Note: default split collapses _____ spaces and strips _____.

```
print("alice,bob,charlie".split(","))
```

Output: _____

```
line = "A101,Laptop,5,29999.99"
parts = line.split(",")
print(parts[0])
print(parts[2])
print(type(parts[2]))
```

`parts[0]` = _____ `parts[2]` = _____ `type(parts[2])` = _____

`.split()` always returns a _____. Every element inside is always a _____. To use `parts[2]` as a number you must write: _____

(d) Replace — predict and run:

```
msg = "hello world"
print(msg.replace("world", "Python"))
print(msg.replace("l", "L"))
print(msg)
```

Does `.replace("l","L")` change only the first `l`? _____

(e) Chaining — because every method returns a string, you can chain calls:

```
raw = "  HELLO, world!  "
print(raw.strip().lower())
```

Trace the two steps: `raw.strip()` → _____ , then `.lower()` → _____

Q18. *(`chr()` and `ord()` — every character is a number.)*

(a) Run and fill in:

```
print(ord('A'))
print(ord('a'))
print(ord('0'))
print(chr(65))
print(chr(97))
print(chr(66))
```

`ord('A')` = _____ `ord('a')` = _____ `ord('0')` = _____ `chr(65)` = _____ `chr(97)` = _____ `chr(66)` = _____

(b) Uppercase letters run from `ord('A')` = 65 to `ord('Z')` = _____. Lowercase letters run from `ord('a')` = 97 to `ord('z')` = _____.

(c) Predict before running:

```
print(chr(ord('A') + 3))
print(chr(ord('z') - 25))
```

Results: _____ · _____

(d) ○ `ord()` returns an `int`. Which Class 3 built-in would convert it to a `float`? _____

Q19. *(Tuples — the formal intro.)*

You've already used tuples: `("Alice", {95, 87, 92})` in Q9, `(0, 0)` in the GPS coords set. Here's the full picture.

(a) Create and index — predict, then run:

```
t = ("Alice", 95, True, 3.14)
print(t[0])
print(t[-1])
print(t[1:3])
print(len(t))
```

Outputs: _____ · _____ · _____ · _____

(b) ○ Immutability — predict first:

```
t[0] = "Bob"
```

Expected error: _____ . Run to confirm. Which other type raises the same error for item assignment? _____

(c) Single-element trap — run both and note the types:

```
print(type((1)))
print(type((1,)))
```

`type((1))` = _____ `type((1,))` = _____ To create a one-element tuple you must write: _____

(d) Tuple vs list — fill in:

- Use a `tuple` when the structure is _____ (fixed fields, like a record)
- Use a `list` when the collection is _____ (elements added, removed, reordered)
- `("Alice", 95, True)` is a _____ because it's a fixed student record
- A class roll of student names is a _____ because names can be added

---

Q20. *(↻ Spaced retrieval — books closed, spans Classes 1–5.)*

No looking back. Write from memory.

(a) `s = "Python"`. What does `s[-3:]` return? What does `s[::-1]` return?

(b) `"name,age,city".split(",")` — what type does this return? What is `result[1]`?

(c) `0.1 + 0.2 == 0.3` — result and one-sentence explanation.

(d) `ord('B')` = ? (recall: `ord('A')` = 65). `chr(122)` = ? (recall: `ord('z')` = 122).

(e) Name all three collection types from Part 1. For each, give the one property that makes it distinct from the other two.

Part 4 — Lists: Indexing, Slicing & Mutation

Q21. *(Indexing and slicing — rapid. Same syntax as strings.)*

```
lst = [10, 20, 30, 40, 50]

print(lst[0])
print(lst[-1])
print(lst[1:3])
print(lst[::-1])
lst[2] = 99
print(lst)
```

(a) Predict all five outputs: _____ · _____ · _____ · _____ · _____

(b) ↻ `s = "hello"`, `s[2] = 'X'` crashes. `lst[2] = 99` works. One word for the property that explains the difference: _____

Q22. *(Mutation methods — append, pop, remove. Predict each output before running.)*

(a) Append:

```
lst = [10, 20, 30]
lst.append(40)
lst.append(50)
print(lst)
```

Output: _____

(b) Pop — removes AND returns the element:

```
lst = [10, 20, 30, 40, 50]
x = lst.pop()
print(x)
print(lst)
y = lst.pop(1)
print(y)
print(lst)
```

`x` = _____ lst after first pop: _____ `y` = _____ lst after second pop: _____

(c) Remove — by value, not by index:

```
lst = [10, 20, 30, 20, 40]
lst.remove(20)
print(lst)
lst.remove(99)
```

After first remove: _____ What happens on the second remove (99 not present)? _____

Q23. *(sort() vs sorted() — the return-value trap.)*

(a) Predict all, then run:

```
nums = [3, 1, 4, 1, 5, 9, 2]

new = sorted(nums)
print(new)
print(nums)

nums.sort()
print(nums)
```

`sorted(nums)` returns: _____ `nums` after `sorted()`: _____ After `nums.sort()`, `nums` is: _____

(b) The trap — predict `result` before running:

```
nums = [3, 1, 4]
result = nums.sort()
print(result)
print(nums)
```

`result` = _____ Why? _____

(c) Fill in:

- `sorted(lst)` — returns a _____, original is _____
- `lst.sort()` — returns _____, mutates _____ in place

Q24. *(Explicit mutable-vs-immutable contrast.)*

Run both blocks:

```
# Block A — string
s = "hello"
s[0] = "H"
```

```
# Block B — list
lst = [30, 10, 20]
lst[0] = 99
lst.sort()
print(lst)
```

Block A: _____ Block B prints: _____

Fill in the contrast table:

```
Operation                          str         list
_____________________________      _________   _________
Change one element in place        ________    ________
Method mutates the original        ________    ________
Method returns a new object        ________    ________
```

Part 5 — Sets: Operations & Why No Indexing

Q25. *(Why no indexing — and the discard/remove distinction.)*

(a) Give the one-sentence explanation that goes deeper than "sets don't support indexing": _____

(b) Run one line at a time — predict which line crashes:

```
s = {"Alice", "Bob", "Charlie"}
s.add("Dana")
print(s)
s.discard("Alice")
print(s)
s.discard("Zara")
print(s)
s.remove("Zara")
print(s)
```

Which line crashes? _____ Error: _____

Difference between `.discard()` and `.remove()` when element is absent: _____

Q26. *(Set algebra — predict, then run. Display order may vary.)*

```
python_devs = {"Alice", "Bob", "Charlie", "Dana"}
ml_devs     = {"Bob", "Dana", "Eve", "Frank"}
```

(a) `python_devs | ml_devs` → _____ *(all unique names from either)*

(b) `python_devs & ml_devs` → _____ *(work on both teams)*

(c) `python_devs - ml_devs` → _____ *(Python only, not on ML)*

(d) `ml_devs - python_devs` → _____ *(is this the same as (c)?)*

(e) Verify that the method form gives identical results:

```
print(python_devs.union(ml_devs))
print(python_devs.intersection(ml_devs))
print(python_devs.difference(ml_devs))
```

Q27. *(Real-world scenario — choose the operation.)*

```
show_a = {"u101", "u102", "u103", "u104", "u105"}
show_b = {"u103", "u105", "u106", "u107"}
```

(a) Users who watched BOTH shows: operation _____ result _____

(b) Show A watchers to recommend Show B to (watched A, not B): operation _____ result _____

(c) Total unique audience (at least one show): operation _____ result _____

(d) New user `"u108"` just watched Show A. Update `show_a` in one line: _____

Part 6 — `in` / `not in`: The Choice of Container IS the Algorithm

Q28. *(Timing demo — run and record your results.)*

```
import time

data_list = list(range(1_000_000))
data_set  = set(range(1_000_000))
target    = 999_999

t0 = time.time()
_ = target in data_list
print(f"list: {(time.time() - t0)*1000:.3f} ms")

t0 = time.time()
_ = target in data_set
print(f"set:  {(time.time() - t0)*1000:.6f} ms")
```

(a) Your results — list: _____ ms set: _____ ms

(b) Approximate ratio (list ÷ set): _____ × slower

(c) Why is `target = 999_999` the worst case for the list specifically? _____

(d) Would `target = 0` make the list fast? What does this tell you about relying on best-case performance? _____

Q29. *(Trade-off table — fill in from memory.)*

Property	list	set	str
in time complexity	_____	_____	_____
Ordered (indexable)	_____	_____	_____
Allows duplicates	_____	_____	_____
Mutable	_____	_____	_____
Elements must be hashable	_____	_____	N/A

Q30. *(Scenario + ⟳ spaced retrieval — books closed.)*

(a) You receive 50,000 banned usernames. Every new signup must be checked. Store in a `list` or a `set`? Justify in two sentences.

(b) ⟳ `" hello world ".split()` — what does this return, and what type are the elements inside?

(c) ⟳ `s = "Python"`. What is `s[-3:]` ? What is `s[::-1]` ?

(d) ⟳ `flagged = {}` in the RapidKart code — what type is `flagged`, and what error does `flagged.add("X")` raise?

(e) ⟳ Write the operator precedence order for: `**`, `+`, `&`, `==`, `not`, `and`, `or`. Then explain why `a & b == 1` is almost always a bug.